

UNIVERSIDADE DE MOGI DAS CRUZES
BACHARELADO EM ENGENHARIA DE SOFTWARE

Felipe Luiz de Franco de Assis 11231504381

Renan Oliveira Tarifa 11231201582

Thainá Esther Soares da Silva 11231100756

Controle e monitoramento de motores
Segurança da informação e LGPD

Mogi das Cruzes

2026

Felipe Luiz de Franco de Assis 11231504381
Renan Oliveira Tarifa 11231201582
Thainá Esther Soares da Silva 11231100756

Controle e monitoramento de motores
Segurança da informação e LGPD

Trabalho apresentado para o Prof. Fabiano Bezerra Menegídio
relacionada à disciplina Segurança da informação do curso
de Bacharelado em Engenharia de Software.

Mogi das Cruzes

2026

RESUMO

O presente trabalho tem como objetivo desenvolver uma aplicação web voltada ao controle e monitoramento de um motor, por meio de um painel de controle interativo. O sistema foi projetado com uma arquitetura de segurança em camadas, contemplando: armazenamento de senhas com hash bcrypt, autenticação em dois fatores (2FA) via e-mail, gerenciamento de sessões com JWT de curta duração e refresh token persistido com hash SHA-256, além de proteção contra tentativas abusivas de acesso por meio de bloqueio temporário de conta. Todas essas medidas visam garantir o acesso restrito e seguro à plataforma, em conformidade com a Lei Geral de Proteção de Dados (LGPD).

Palavras-chave: Aplicação Web, Autenticação, Autenticação em dois Fatores, JWT, bcrypt, Painel de Controle, Monitoramento, LGPD.

ABSTRACT

This work aims to develop a web application for motor control and monitoring through an interactive control panel. The system was designed with a layered security architecture, encompassing: password storage with bcrypt hashing, two-factor authentication (2FA) via e-mail, session management with short-lived JWT and refresh token persisted with SHA-256 hash, as well as protection against abusive access attempts through temporary account lockout. All these measures aim to ensure restricted and secure access to the platform, in compliance with Brazil's General Data Protection Law (LGPD).

Keywords: Web Application, Authentication, Two-Factor Authentication, JWT, bcrypt, Control Panel, Monitoring, LGPD.

LISTA DE FIGURAS

Figura 1 – Diagrama geral do sistema

Figura 2 - Estrutura interna do BackEnd

Figura 3 - Fluxo geral de autenticação e gerenciamento de sessão

Figura 4 – Fluxograma da autentificação com senha

Figura 5 – Fluxograma da validação de código 2FA

Figura 6 – Fluxograma de fluxo de sessão

Figura 7 – Fluxograma do Logout

Figura 8 – Fluxograma do código de recuperação

Figura 9 – Fluxograma de redefinição da senha

Figura 10 – Diagrama de comunicação de Banco de Dados

Figura 11 – Diagrama comunicação entre FrontEnd e BackEnd

Figura 12 – Tela de criação de conta com campo de aceite da Política de Privacidade

Figura 13 – Credenciais inválidas

Figura 14 - Conta bloqueada após 3 credenciais inválidas

Figura 15 – Solicitação de código de acesso

Figura 16 – Envio de código de acesso ao e-mail

Figura 17 – Tentativa de login com token expirado

Figura 18 - Tela de perfil do usuário com opções de direitos LGPD

Figura 19 - E-mail de exportação dos dados pessoais conforme a LGPD

Figura 20 - Modal de confirmação de exclusão de conta e dados

Figura 21 – Representação da saída do log

Figura 22 - Resposta da API no cadastro de usuário

Figura 23 - Resposta de bloqueio de conta

Figura 24 - Validação de token de autenticação

SUMÁRIO

1. INTRODUÇÃO	14
2. ARQUITETURA E FLUXOS.....	15
2.1 Arquitetura geral do sistema	15
2.2 Fluxo de autenticação.....	17
2.3 Fluxo de sessão	19
2.4 Fluxo de recuperação de senha.....	20
2.5 Estruturação de Banco de dados	21
2.6 Comunicação entre FrontEnd, BackEnd e banco	22
2.7 Termo de consentimento e conformidade com a LGPD	23
2.8 Identificação dos ativos do sistema	24
3. ANÁLISE DE RISCOS E AMEAÇAS.....	26
3.1 identificação das principais ameaças	26
3.2 Impacto e probabilidade dos riscos	27
3.3 Vulnerabilidades encontradas.....	27
3.4 Contramedidas adotadas pelo sistema	27
4. TESTES DE SEGURANÇA DOCUMENTADOS.....	29
4.1 Metodologia dos testes.....	29
4.2 Testes executados.....	29
5. FUNDAMENTAÇÃO CIENTIFICA	36
5.1 Conceitos de autenticação e autorização	36
5.2 Funcionamento do JWT.....	36
5.3 Criptografia de senhas com bcrypt	36
5.4 Conceitos de 2FA/MFA.....	37
5.5 Segurança em APIs REST	37
5.6 LGPD e proteção de dados.....	37
5.7 Referências acadêmicas, artigos e normas técnicas.....	38
6. REFERÊNCIAS.....	39

1. INTRODUÇÃO

A crescente digitalização de processos industriais e o aumento de sistemas conectados à internet elevaram significativamente os riscos relacionados à segurança da informação. Nesse cenário, aplicações web que gerenciam dispositivos físicos, como motores e sensores, tornam-se alvos sensíveis, exigindo mecanismos robustos de proteção tanto no acesso quanto no tratamento dos dados.

Diante disso, o presente trabalho propõe o desenvolvimento de uma aplicação web voltada ao controle e monitoramento de um motor, por meio de um painel de controle interativo. O sistema foi projetado com uma arquitetura de segurança em camadas, contemplando: armazenamento de senhas com hash bcrypt, autenticação em dois fatores (2FA) via e-mail, gerenciamento de sessões com JWT de curta duração e refresh token persistido com hash SHA-256, além de proteção contra tentativas abusivas de acesso por meio de bloqueio temporário de conta.

A escolha dessas tecnologias se justifica pela necessidade de garantir um ambiente seguro e confiável, dificultando acessos não autorizados e assegurando o tratamento adequado dos dados dos usuários em conformidade com a Lei Geral de Proteção de Dados (LGPD). Assim, o projeto busca unir funcionalidade e segurança, entregando uma solução moderna, eficiente e responsável no contexto de aplicações web conectadas a dispositivos físicos.

2. ARQUITETURA E FLUXOS

2.1 Arquitetura geral do sistema

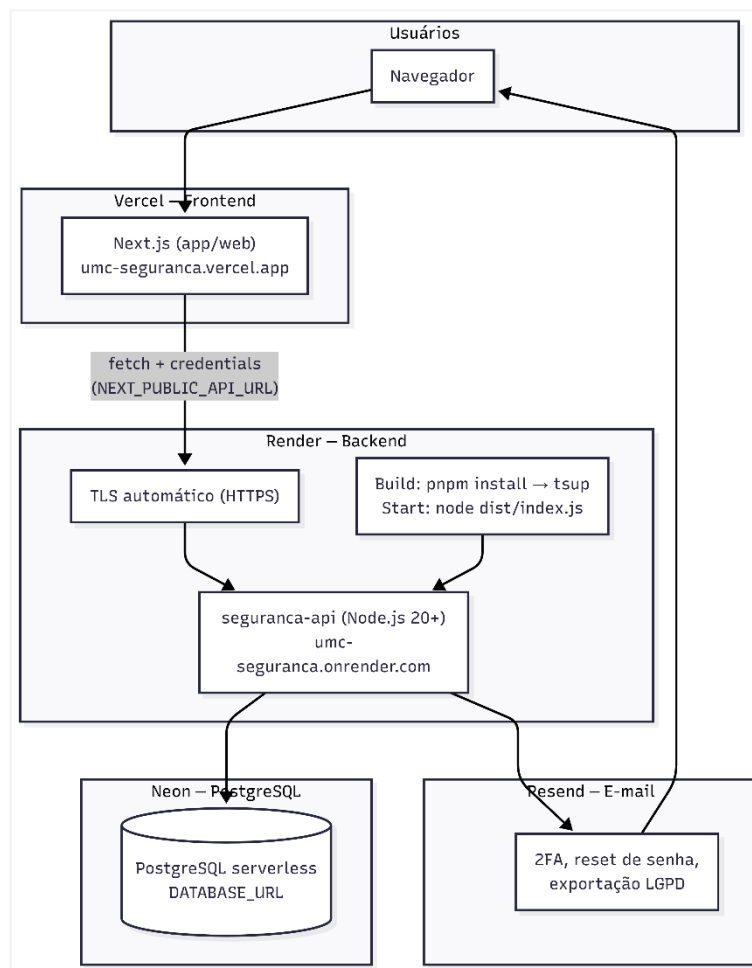


Figura 1 – Diagrama geral do sistema

A aplicação é estruturada seguindo o modelo cliente-servidor, com separação clara entre frontend, backend e banco de dados, cada camada hospedada de forma independente em serviços de nuvem especializados.

O frontend foi desenvolvido com Next.js e TypeScript, hospedado na plataforma Vercel. A interface é composta por páginas dedicadas ao login, cadastro de usuário e painel de controle do motor, além de componentes reutilizáveis organizados na pasta `components/ui`. A comunicação com o backend é centralizada no módulo `lib/api.ts`, e o estado de autenticação do usuário é gerenciado globalmente por meio do contexto `auth-context.tsx`.

O backend foi desenvolvido com o framework Fastify e TypeScript, hospedado na plataforma render. A aplicação expõe uma API REST organizada em rotas versionadas sob o prefixo `/v1`, contemplando os módulos de autenticação, sessões e usuários. A validação de todas as entradas é realizada com a biblioteca Zod, e a documentação da API é gerada automaticamente no padrão OpenAPI e disponibilizada via Scalar no endpoint `/docs`. Para garantir a segurança da aplicação, são utilizados os plugins Helmet, responsável pela definição de cabeçalhos HTTP de segurança, CORS, configurado para aceitar requisições apenas de origens

autorizadas, e Rate Limit, que restringe o número de requisições por janela de tempo, protegendo a API contra ataques de força bruta e abuso.

O banco de dados utilizado é o PostgreSQL, provisionado na plataforma Neon Database. O acesso e a manipulação dos dados são realizados por meio do Drizzle ORM, que define o schema das tabelas diretamente em TypeScript. O banco é composto por três tabelas principais: `users`, que armazena os dados dos usuários com o campo `name` criptografado em nível de aplicação antes da persistência, em conformidade com a LGPD; `access_codes`, que registra os códigos temporários utilizados tanto na autenticação de dois fatores quanto na recuperação de senha, com o token também armazenado de forma criptografada; e `sessions`, que gerencia as sessões ativas dos usuários por meio do hash SHA-256 do Refresh Token, além dos campos `expiresAt` e `revokedAt` para controle de validade e revogação.

O envio de e-mails transacionais, tanto o código de autenticação de dois fatores quanto o código de recuperação de senha são realizados pelo serviço Resend, integrado ao backend por meio da biblioteca oficial.

No momento do cadastro, o sistema registra o consentimento explícito do usuário por meio do campo `consentAt`, que armazena a data e hora em que o aceite foi realizado. Caso o consentimento não seja fornecido, o campo é persistido como nulo, permitindo que o sistema diferencie usuários que aceitaram os termos daqueles que não o fizeram, em conformidade com os requisitos da LGPD.

Estrutura interna do Backend

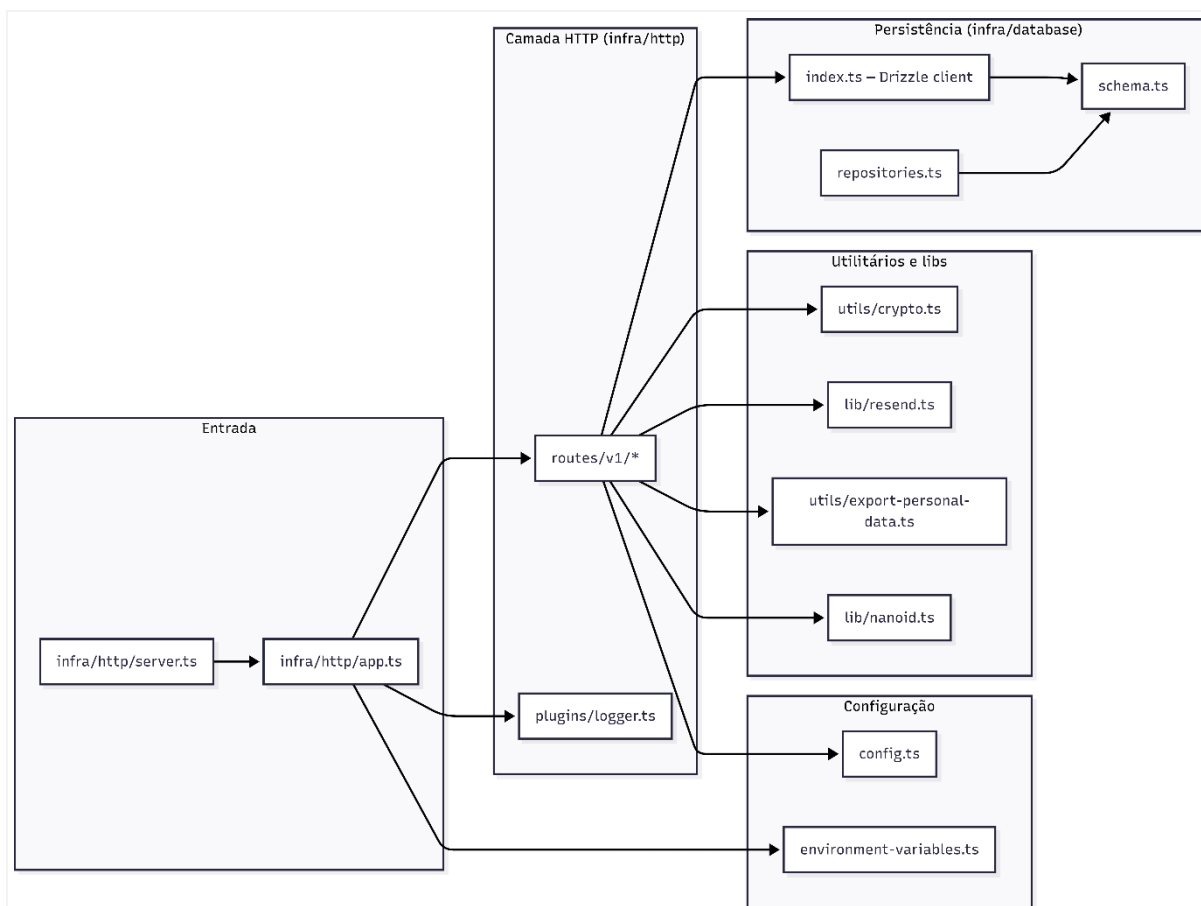


Figura 2 - Estrutura interna do Backend

O BackEnd da aplicação foi estruturado em camadas organizadas por responsabilidade, visando facilitar a manutenção, reutilização de código e separação das funcionalidades do sistema. A aplicação utiliza o framework Fastify para gerenciamento das requisições HTTP, com rotas organizadas em módulos responsáveis pelos fluxos de autenticação, sessões e usuários.

A comunicação com o banco de dados PostgreSQL é realizada por meio do Drizzle ORM, enquanto módulos utilitários concentram funcionalidades como criptografia, geração de tokens e envio de e-mails. A Figura 2 apresenta a organização interna das principais camadas e componentes do backend da aplicação.

2.2 Fluxo de autenticação

Esta seção descreve o processo de autenticação da aplicação, estruturado em duas etapas sequenciais. A primeira etapa consiste na validação das credenciais do usuário por meio de e-mail e senha. A segunda etapa aplica um fator adicional de verificação por meio de um código temporário enviado ao e-mail cadastrado, garantindo que somente o titular da conta possa concluir o acesso ao sistema.

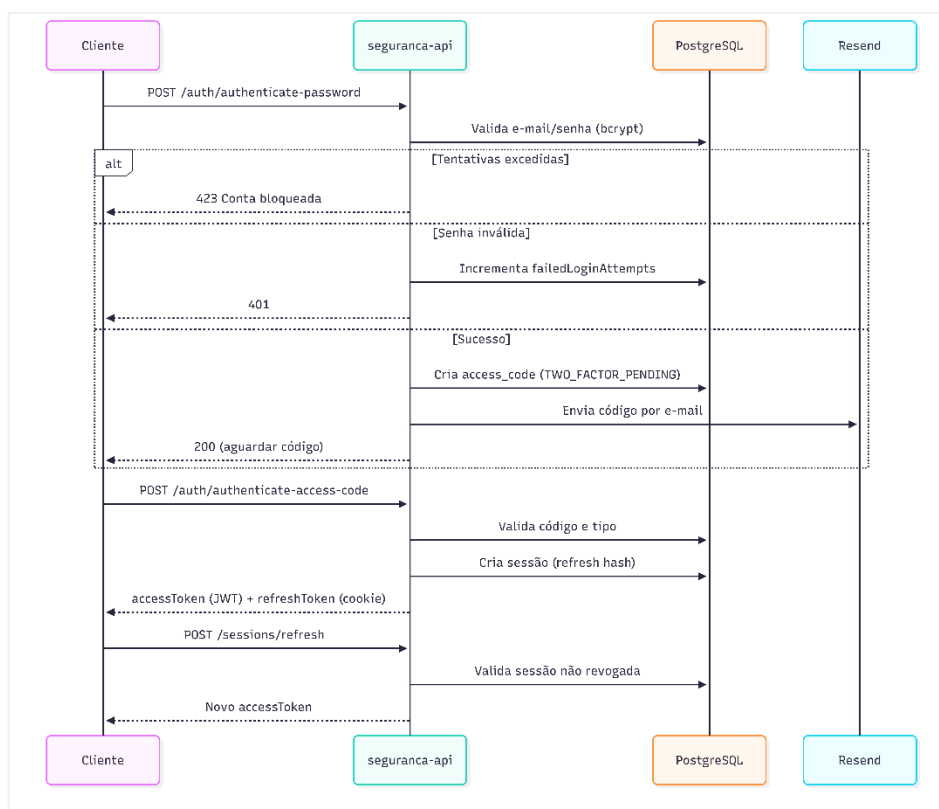


Figura 3 - Fluxo geral de autenticação e gerenciamento de sessão

Autenticação com senha

O fluxo de autenticação tem início quando o usuário submete suas credenciais, e-mail e senha, por meio da interface web. O sistema realiza uma consulta ao banco de dados para verificar a existência do usuário cadastrado. Caso o usuário não seja encontrado, a requisição é encerrada com o código HTTP 400. Estando o usuário presente, o sistema verifica se a conta se encontra temporariamente bloqueada,

condição determinada pelo campo `lockedUntil`, retornando HTTP 423 em caso positivo. Superadas essas verificações, a senha fornecida é comparada ao hash armazenado por meio da função `bcrypt.compare()`. Em caso de falha, o contador de tentativas frustradas é incrementado e, ao atingir o limite de 3 tentativas, a conta é bloqueada temporariamente por 15 minutos. Sendo a senha válida, o sistema zera o contador de tentativas falhas, remove eventuais códigos 2FA anteriores vinculados ao usuário e prossegue para a geração do código de verificação.

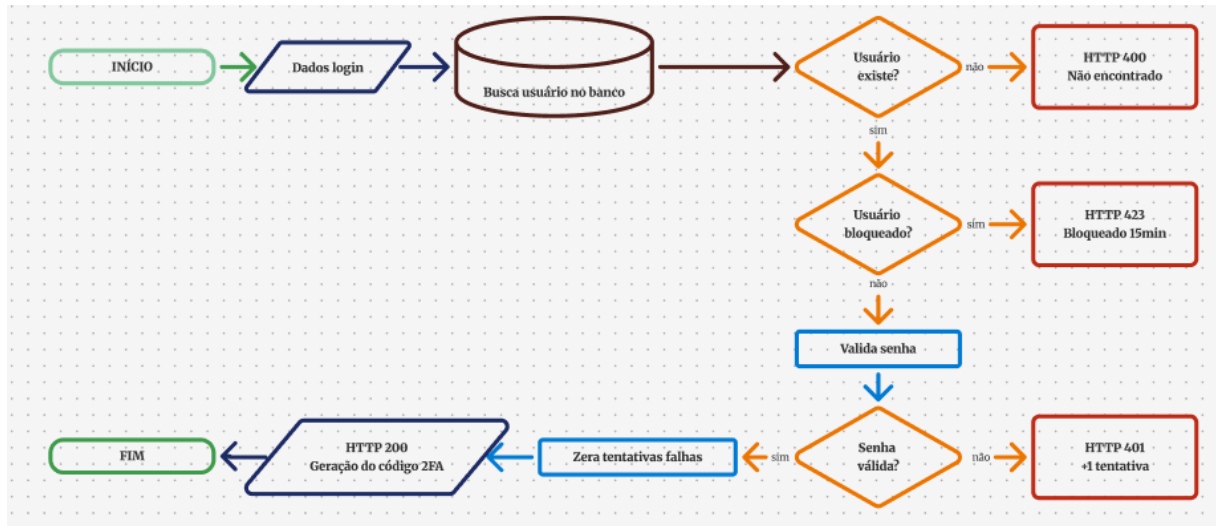


Figura 4 – Fluxograma da autenticação com senha

Geração e validação do código 2FA

Validada a senha com sucesso, o sistema gera um código numérico de seis dígitos por meio da função `generateNanoid (6)`, que é persistido no banco de dados associado ao identificador do usuário com o tipo `TWO_FACTOR_EMAIL`. Em seguida, o código é enviado ao endereço de e-mail cadastrado por meio do serviço Resend, com tempo de validade de 10 minutos definido pela constante `two_factor_peding_ttl_min`. Nenhuma sessão é criada neste momento, o acesso à plataforma permanece condicionado à validação do código recebido, e a resposta HTTP 200 sinaliza ao cliente que o código foi enviado com sucesso.

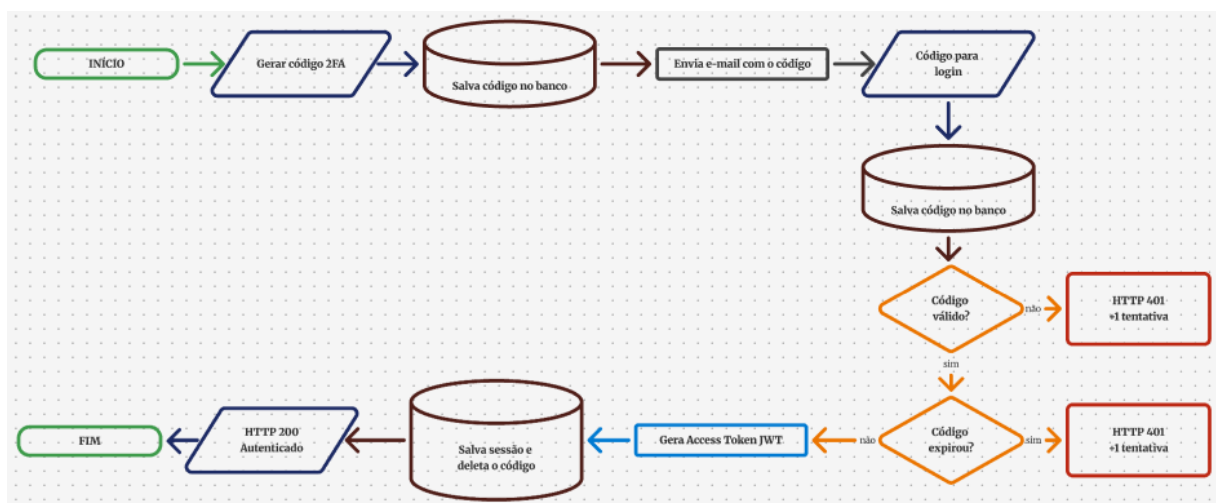


Figura 5 – Fluxograma da validação com código 2FA

Na segunda etapa, o usuário submete o código de seis dígitos recebido por e-mail. O sistema realiza duas verificações: a correspondência entre o código informado e o armazenado, e a validade temporal com base no campo `createdAt`. Caso o código seja inválido ou tenha expirado, a requisição é rejeitada com HTTP 401 e o contador de tentativas é incrementado, podendo bloquear a conta após 3 tentativas falhas. Sendo o código válido, o sistema gera um Access Token JWT com expiração de 15 minutos e um Refresh Token JWT com validade de 7 dias. O Refresh Token é armazenado no banco em formato hash SHA-256, garantindo que o valor original nunca seja persistido diretamente. O Access Token é retornado no corpo da resposta, enquanto o Refresh Token é definido como cookie `HttpOnly`, impedindo seu acesso por JavaScript. O código de acesso utilizado é então excluído do banco de dados e a sessão é registrada para gerenciamento futuro.

2.3 Fluxo de sessão

Esta seção descreve o ciclo de vida da sessão do usuário após a autenticação. O gerenciamento de sessão contempla dois momentos distintos: a renovação automática do token de acesso por meio do mecanismo de refresh token, que mantém o usuário autenticado sem a necessidade de novo login, e o encerramento explícito da sessão por meio do logout, que revoga o acesso de forma segura no servidor.

Renovação de sessão (refresh token)

O gerenciamento de sessão da aplicação utiliza dois tokens: o Access Token JWT, com duração de 15 minutos, responsável pela autenticação das requisições, e o Refresh Token, com validade de 7 dias, armazenado como cookie `HttpOnly` e persistido no banco em formato hash SHA-256.

Quando o Access Token expira, o cliente realiza uma requisição ao endpoint `/sessions/refresh`, enviando o Refresh Token via cookie. O sistema valida a sessão, verifica a integridade do token e confirma sua validade. Caso as verificações sejam aprovadas, uma nova sessão é criada com um novo par de tokens, revogando automaticamente a sessão anterior. O novo Access Token é retornado na resposta, enquanto o novo Refresh Token substitui o anterior no cookie `HttpOnly`.

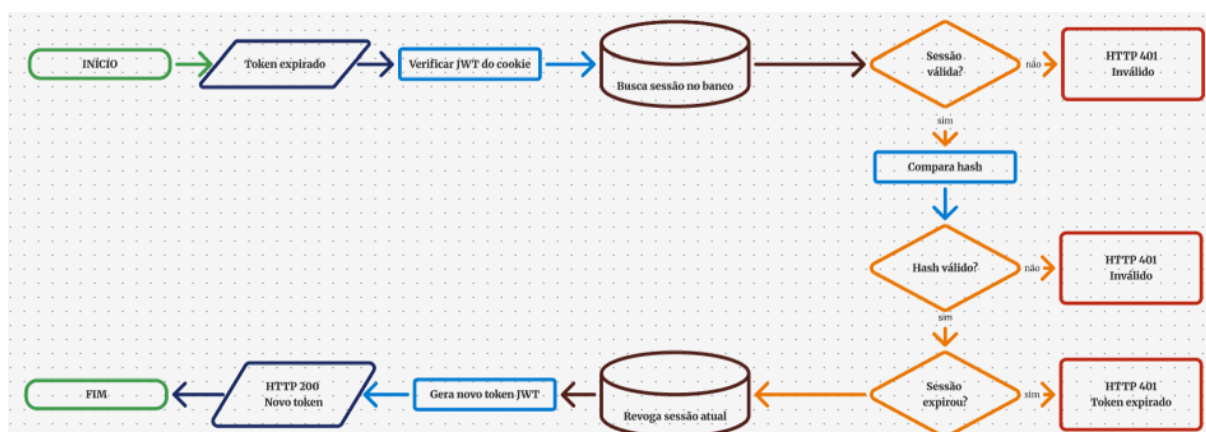


Figura 6 – Fluxograma de fluxo de sessão

Logout

O encerramento de sessão é iniciado pelo usuário por meio de uma ação explícita na interface. O frontend envia uma requisição ao endpoint POST /logout, e o sistema identifica a sessão atual por meio do cookie refreshToken, extraíndo o identificador da sessão contido no payload JWT via jwtVerify(). Localizada a sessão no banco de dados, o campo revokedAt é preenchido com a data e hora correntes, marcando-a como revogada. A partir desse momento, o Refresh Token associado torna-se inválido e não pode ser utilizado para renovação de sessão. O frontend então limpa o Access Token em memória e redireciona o usuário para a tela de login. A operação é concluída com retorno HTTP 200, confirmando o encerramento seguro do acesso à plataforma.

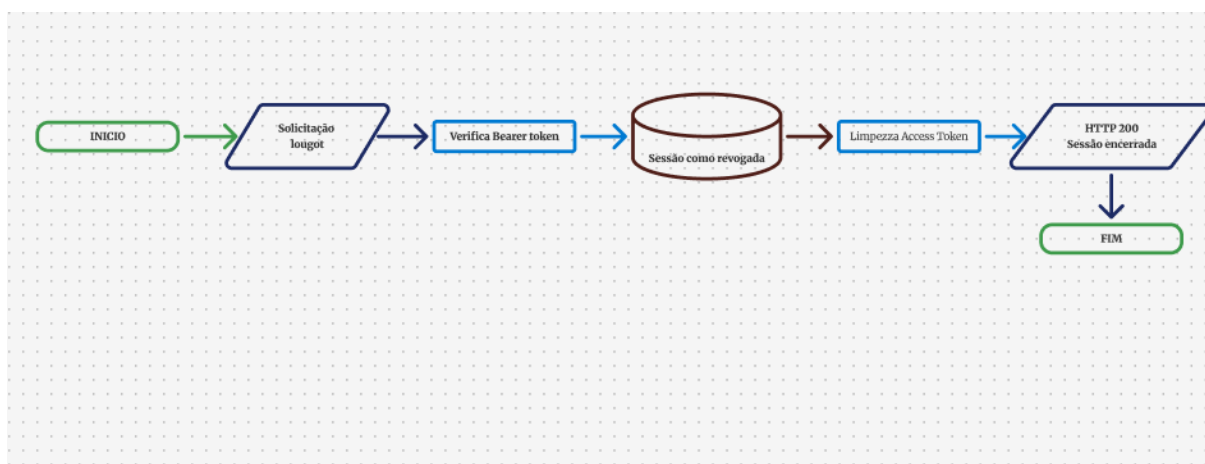


Figura 7 – Fluxograma do Logout

2.4 Fluxo de recuperação de senha

Esta seção descreve o processo de recuperação de senha, disponível ao usuário em caso de perda ou esquecimento das suas credenciais. O fluxo é independente do processo de autenticação em duas etapas e é dividido em duas etapas: a solicitação de um token de recuperação enviado por e-mail e a redefinição da senha por meio desse token.

Solicitação do código de recuperação

O fluxo de recuperação de senha tem início quando o usuário informa seu endereço de e-mail por meio da interface web. O sistema realiza uma consulta ao banco de dados para verificar a existência do usuário cadastrado. Caso não seja encontrado, a requisição é encerrada com HTTP 400. Sendo o usuário localizado, o sistema gera um token alfanumérico de 6 caracteres por meio da função generateNanoid(6), que é persistido na tabela access_code com o tipo PASSWORD_RESET. Em seguida, o token é enviado ao endereço de e-mail cadastrado por meio do serviço Resend, com validade de 60 minutos definida pela constante PASSWORD_RESET_TTL_MIN. A resposta HTTP 200 confirma ao cliente que o código foi enviado com sucesso.

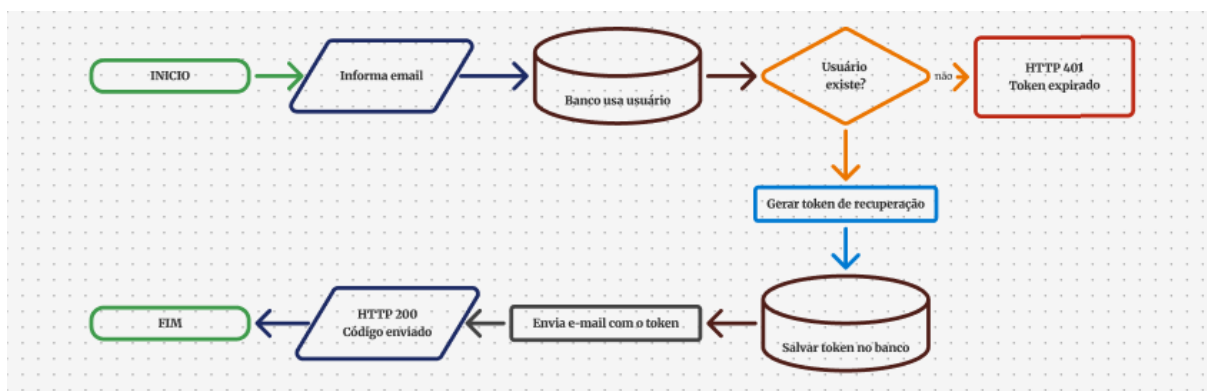


Figura 8 – Fluxograma do código de recuperação

Redefinição da senha

Na segunda etapa, o usuário informa o token recebido por e-mail e a nova senha, que deve conter entre 8 e 128 caracteres. O sistema localiza o registro correspondente na tabela `access_code` pelo token informado. Caso o token não seja encontrado, a requisição é rejeitada com HTTP 401. Se o token existir, mas tiver ultrapassado o prazo de 60 minutos, a requisição também é rejeitada com HTTP 401 e mensagem de token expirado. Sendo o token válido, o sistema gera um novo hash bcrypt da nova senha e atualiza o registro do usuário no banco, zerando também o contador de tentativas falhas e removendo qualquer bloqueio temporário ativo. Por fim, todas as sessões ativas do usuário são revogadas, forçando um novo login em todos os dispositivos. A operação é concluída com retorno HTTP 200, confirmando a redefinição segura da senha.

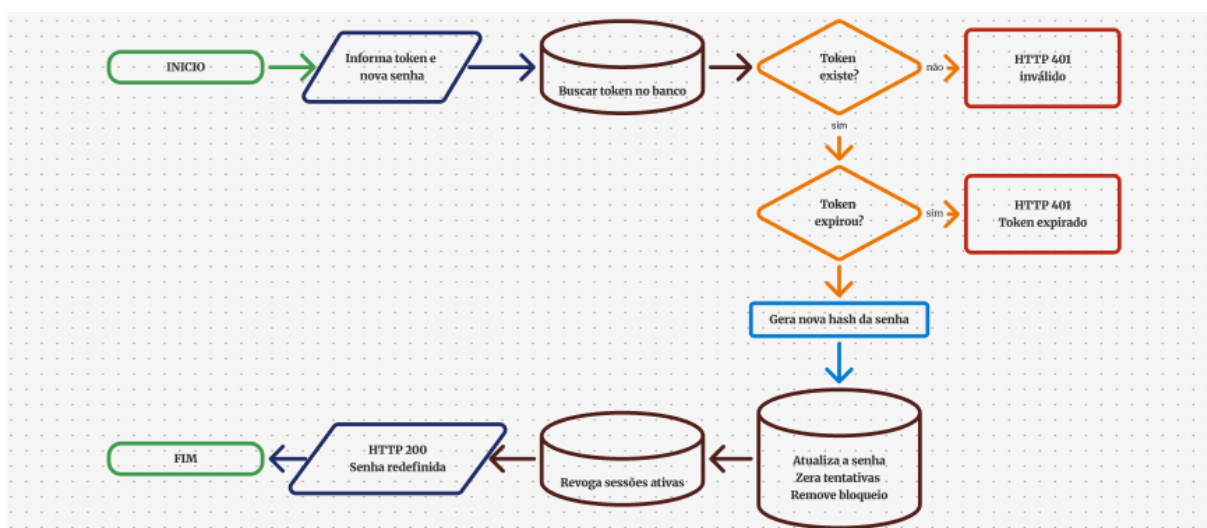


Figura 9 – Fluxograma de redefinição da senha

2.5 Estruturação de Banco de dados

O banco de dados da aplicação é composto por três tabelas principais, responsáveis pelos fluxos de autenticação, gerenciamento de sessão e recuperação de senha. Os dados sensíveis são protegidos por mecanismos de criptografia e hash, garantindo maior segurança e conformidade com a LGPD.

A tabela `users` armazena os dados cadastrais dos usuários, incluindo senha criptografada com `bcrypt`, controle de tentativas de login e bloqueio temporário de conta. Também registra o consentimento do usuário por meio do campo `consentAt`.

A tabela `access_codes` é responsável pelo armazenamento dos códigos temporários utilizados na autenticação em dois fatores e recuperação de senha, enquanto a tabela `sessions` gerencia as sessões autenticadas dos usuários, armazenando o hash SHA-256 do Refresh Token e informações relacionadas à validade e revogação da sessão.

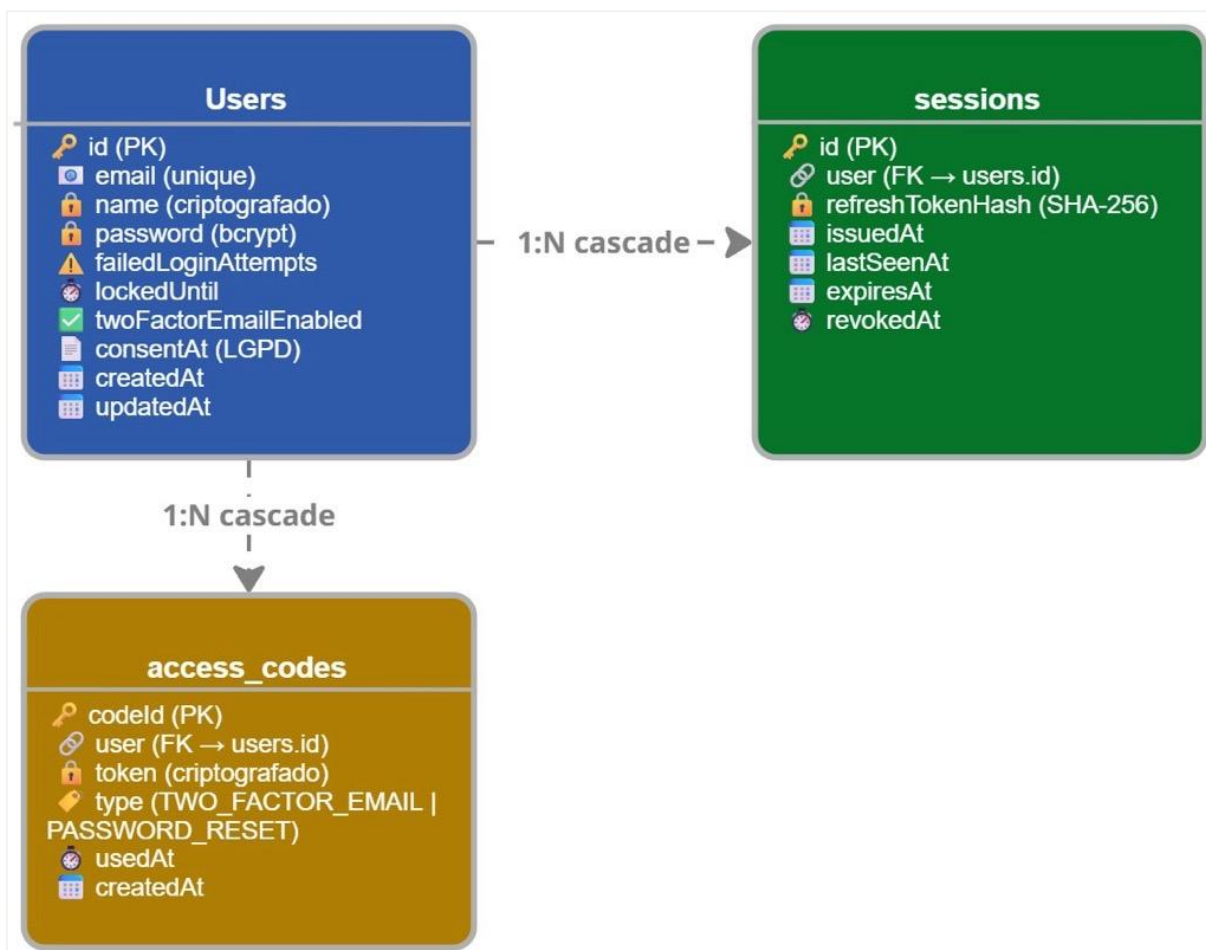


Figura 10 – Diagrama de comunicação de Banco de Dados

2.6 Comunicação entre FrontEnd, BackEnd e banco

A comunicação entre o frontend e o backend é realizada por meio de requisições HTTP centralizadas na função `apiRequest`, localizada no módulo `lib/api.ts`. Essa função utiliza a `Fetch API` nativa do navegador e é configurada para enviar automaticamente os cookies em todas as requisições por meio da opção `credentials: 'include'`, o que garante que o cookie `HttpOnly` contendo o Refresh Token seja transmitido ao servidor sem necessidade de manipulação explícita pelo código da aplicação. A URL base da API é definida pela variável de ambiente `NEXT_PUBLIC_API_URL`, apontando para o serviço hospedado na plataforma `render`. Em caso de erro de rede ou resposta com status de falha, a função retorna

uma estrutura padronizada contendo o campo error, permitindo que cada página trate os erros de forma consistente.

O estado de autenticação do usuário é gerenciado globalmente por meio do contexto AuthContext, implementado com a Context API do React. O Access Token recebido após a validação do código 2FA é armazenado exclusivamente em memória por meio do estado accessToken, nunca sendo persistido em localStorage ou sessionStorage, o que reduz a superfície de ataque a scripts maliciosos. O valor booleano isAuthenticated é derivado diretamente da presença do token em memória, e a função logout limpa esse estado, encerrando o acesso do usuário à interface sem necessidade de recarregar a página. Toda a aplicação acessa essas informações por meio do hook useAuth, que garante que o contexto seja utilizado apenas dentro do escopo do AuthProvider.

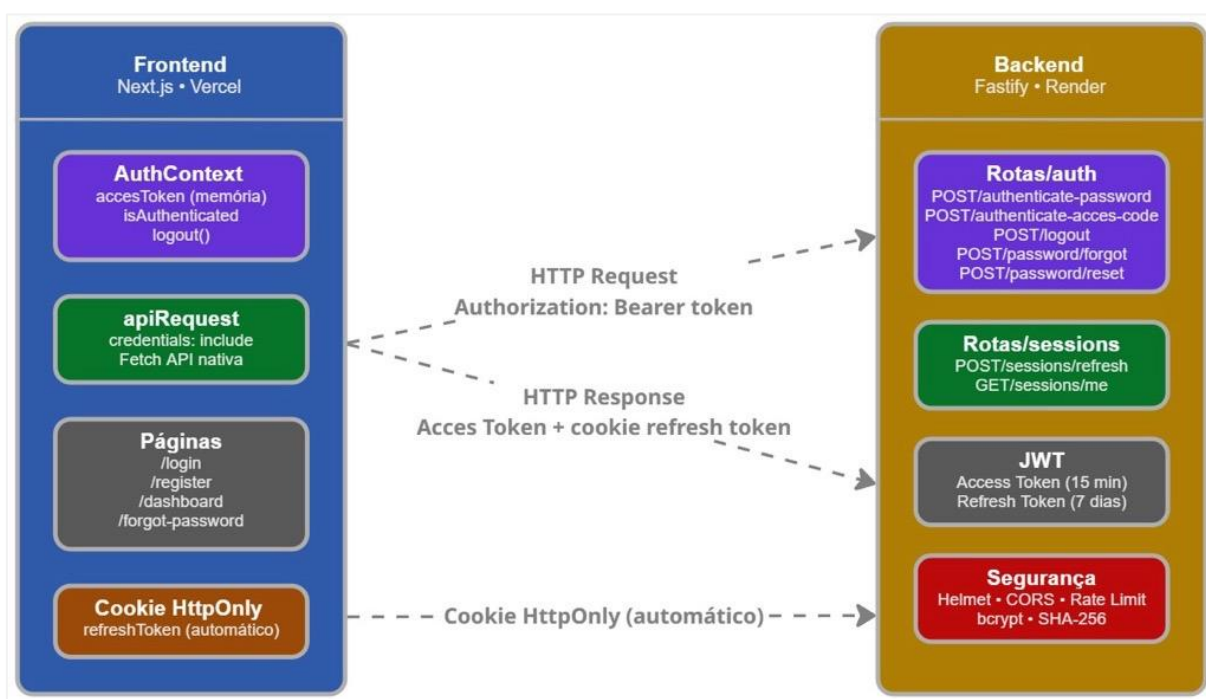


Figura 11 – Diagrama comunicação entre FrontEnd e BackEnd

2.7 Termo de consentimento e conformidade com a LGPD

A Lei Geral de Proteção de Dados (LGPD), instituída pela Lei nº 13.709/2018, estabelece que o tratamento de dados pessoais deve ser fundamentado em bases legais específicas, sendo o consentimento do titular uma das principais. Nesse contexto, a aplicação implementa um mecanismo de registro de consentimento no fluxo de cadastro de usuário.

Durante o registro, o usuário deve preencher nome, e-mail e senha, e declarar que leu e concorda com o tratamento dos seus dados conforme a Política de Privacidade, por meio de uma caixa de seleção obrigatória exibida na tela de criação de conta, conforme ilustrado na Figura 7. Sem essa confirmação, o botão de cadastro permanece inativo, impedindo o prosseguimento do registro. Esse consentimento é representado pelo parâmetro consent no corpo da requisição POST /users, e caso seja concedido, o sistema registra a data e hora exata do aceite no campo consentAt

da tabela users. Caso contrário, o campo é persistido como nulo, garantindo rastreabilidade e diferenciação entre usuários que aceitaram ou não os termos.

Além do registro de consentimento, a aplicação adota outras medidas alinhadas à LGPD. O campo name do usuário é criptografado em nível de aplicação antes de ser persistido no banco de dados, protegendo dados pessoais identificáveis contra exposição indevida em caso de acesso não autorizado ao banco. As senhas são armazenadas exclusivamente em formato de hash bcrypt, nunca em texto puro. O tratamento de dados é restrito ao mínimo necessário para o funcionamento do sistema, seguindo o princípio da minimização de dados previsto na lei.

A interface da aplicação disponibiliza ao usuário autenticado uma tela de perfil que centraliza as funcionalidades relacionadas aos seus direitos previstos na LGPD. Nessa tela, o usuário pode visualizar seus dados cadastrais, solicitar a exportação completa dos dados vinculados à sua conta em formato CSV enviada ao e-mail cadastrado, conforme os artigos 18, I e II da lei, e solicitar a exclusão definitiva da conta e de todos os dados associados, conforme o artigo 18, VI da lei. Essa implementação demonstra o compromisso da aplicação com a transparência e com o respeito aos direitos do titular de dados pessoais.

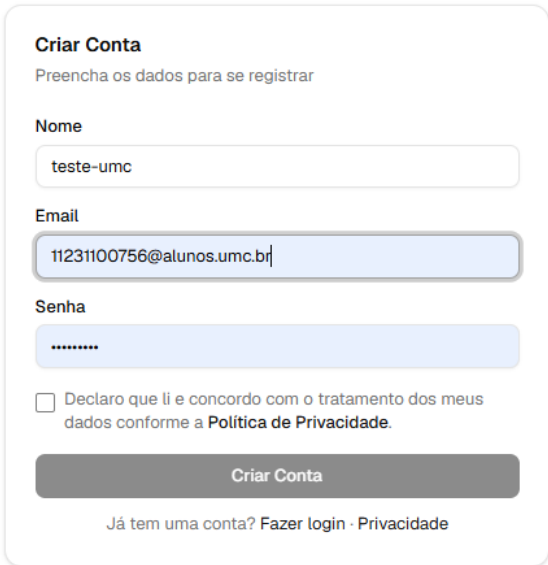


Figura 12 – Tela de criação de conta com campo de aceite da Política de Privacidade

2.8 Identificação dos ativos do sistema

Os ativos do sistema representam os recursos essenciais para o funcionamento da aplicação e para a segurança das informações processadas. A identificação desses ativos é importante para compreender quais elementos devem receber maior nível de proteção, monitoramento e controle, especialmente em relação aos riscos de segurança da informação e conformidade com a LGPD.

A aplicação possui ativos distribuídos entre infraestrutura, autenticação, dados sensíveis e serviços externos integrados ao sistema. Cada ativo desempenha um papel específico no funcionamento da plataforma e possui impacto direto na disponibilidade, integridade e confidencialidade das informações.

FrontEnd da aplicação

O FrontEnd desenvolvido em Next.js representa a camada de interação entre o usuário e o sistema. Esse componente é responsável pela exibição das interfaces de autenticação, gerenciamento de conta, painel de controle do motor e funcionalidades relacionadas à LGPD.

API BackEnd

O BackEnd desenvolvido com Fastify centraliza as regras de negócio, autenticação, gerenciamento de sessões, validação de tokens e comunicação com o banco de dados. Esse ativo é considerado um dos componentes mais críticos da aplicação, pois processa diretamente informações sensíveis dos usuários.

Banco de dados PostgreSQL

O banco de dados armazena informações relacionadas aos usuários, sessões autenticadas, códigos temporários e registros de consentimento da LGPD. Por conter dados pessoais e informações de autenticação, esse ativo exige mecanismos elevados de proteção e controle de acesso.

Tokens e sessões de autenticação

Os Access Tokens JWT e Refresh Tokens utilizados pela aplicação representam ativos críticos de segurança, pois são responsáveis pelo controle de acesso dos usuários autenticados. O comprometimento desses tokens poderia permitir acesso indevido às funcionalidades protegidas do sistema.

Serviço de envio de e-mails

O serviço Resend é utilizado para envio de códigos 2FA, recuperação de senha e exportação de dados pessoais. Esse componente é essencial para o funcionamento dos mecanismos de autenticação e conformidade com a LGPD.

Dados pessoais dos usuários

Os dados pessoais armazenados pela aplicação, como nome, e-mail e registros de consentimento, representam ativos sensíveis protegidos pela LGPD. Esses dados recebem tratamento com mecanismos de criptografia, hash e controle de acesso para reduzir riscos de exposição indevida.

A tabela a seguir apresenta a classificação dos principais ativos identificados na aplicação, considerando sua função no sistema e o nível de criticidade relacionado à segurança da informação e proteção dos dados processados. A identificação desses ativos auxilia na definição das medidas de proteção e das contramedidas adotadas pela aplicação.

3. ANÁLISE DE RISCOS E AMEAÇAS

A análise de riscos e ameaças tem como objetivo identificar os principais vetores de ataque aos quais a aplicação está exposta, avaliar o impacto e a probabilidade de cada ameaça, mapear vulnerabilidades existentes e descrever as contramedidas adotadas pelo sistema para mitigá-las. A segurança da aplicação foi planejada desde a fase de desenvolvimento, incorporando controles técnicos diretamente no código para reduzir a superfície de ataque e proteger os dados dos usuários.

3.1 Identificação das principais ameaças

Esta seção descreve as principais ameaças identificadas para o contexto da aplicação, considerando seu modelo de autenticação, gerenciamento de sessão e exposição de endpoints via API REST.

Brute force

O ataque de força bruta consiste na tentativa sistemática e automatizada de descobrir credenciais válidas por meio da submissão repetida de combinações de senhas. Em aplicações web, esse tipo de ataque é especialmente direcionado aos endpoints de autenticação, onde o atacante utiliza ferramentas automatizadas para testar um grande volume de senhas em sequência.

Session hijacking

O sequestro de sessão ocorre quando um atacante obtém acesso ao token de sessão de um usuário legítimo e o utiliza para se autenticar no sistema em seu lugar. Esse ataque pode ser realizado por meio de interceptação de tráfego, acesso físico ao dispositivo da vítima ou exploração de vulnerabilidades em scripts maliciosos que conseguem ler tokens armazenados de forma insegura no navegador.

SQL injection

A injeção de SQL consiste na inserção de comandos SQL maliciosos em campos de entrada da aplicação, com o objetivo de manipular as consultas executadas no banco de dados. Um ataque bem-sucedido pode permitir acesso não autorizado a dados, modificação ou exclusão de registros e até comprometimento total do banco de dados. No contexto da aplicação, o banco de dados utilizado é o PostgreSQL, provisionado na plataforma Neon database. Embora o PostgreSQL possua mecanismos internos de segurança e controle de privilégios mais robustos em comparação a outros bancos relacionais, a ameaça de injeção SQL permanece relevante caso as consultas sejam construídas de forma insegura, sem o uso de parâmetros vinculados ou abstrações adequadas.

Phishing

O phishing é uma técnica de engenharia social na qual o atacante induz o usuário a fornecer suas credenciais por meio de páginas falsas que simulam a interface legítima da aplicação. No contexto da autenticação em duas etapas, o phishing pode ser utilizado para capturar tanto a senha quanto o código 2FA enviado por e-mail, caso o usuário seja enganado a tempo.

Vazamento de tokens

O vazamento de tokens ocorre quando os tokens de autenticação, como o Access Token JWT ou o Refresh Token, são expostos de forma indevida. Isso pode acontecer por armazenamento inseguro no navegador, exposição em logs do servidor, transmissão em canais não criptografados ou acesso por scripts maliciosos à memória da aplicação.

Ataques de replay

Um ataque de replay ocorre quando um atacante intercepta uma requisição autenticada válida e a reenvia ao servidor para obter acesso não autorizado. Esse tipo de ataque explora tokens ou códigos que ainda estão dentro do prazo de validade, buscando reutilizá-los de forma ilegítima.

3.2 Impacto e probabilidade dos riscos

A tabela a seguir apresenta a avaliação de impacto e probabilidade de cada ameaça identificada, considerando os controles de segurança já implementados no sistema.

Ameaça	Probabilidade	Impacto	Nível de riscos
Brute force	Baixa	Alto	Médio
Session hijacking	Baixa	Alto	Médio
SQL injection	Muito Baixa	Alto	Baixo
Phishing	Média	Alto	Alto
Vazamento de tokens	Baixa	Alto	Médio
Ataques de replay	Baixa	Médio	Baixo

Tabela 1 – Identificação do impacto e probabilidade dos riscos

A probabilidade de cada ameaça foi avaliada considerando os controles técnicos implementados. O phishing representa o maior nível de risco residual por ser um vetor de ataque que depende do comportamento do usuário e não pode ser completamente mitigado por controles técnicos no servidor.

3.3 Vulnerabilidades encontradas

Durante o desenvolvimento e os testes realizados diretamente pela interface de documentação interativa da API, disponibilizada via Swagger no endpoint /docs, não foram identificadas vulnerabilidades críticas no sistema. Os testes de autenticação com credenciais inválidas e tokens expirados retornaram os códigos de erro esperados, confirmando o funcionamento correto dos mecanismos de proteção implementados. Em todos os cenários testados, o sistema negou o acesso de forma adequada, sem expor informações sensíveis nas mensagens de erro retornadas pela API.

3.4 Contramedidas adotadas pelo sistema

O sistema implementa um conjunto de contramedidas técnicas para mitigar as ameaças identificadas. Contra-ataques de força bruta, o sistema limita a 3 o número de tentativas de login malsucedidas, bloqueando temporariamente a conta por 15

minutos após esse limite ser atingido. Esse controle é aplicado tanto na etapa de validação de senha quanto na validação do código 2FA.

Para mitigar o sequestro de sessão, o Refresh Token é armazenado exclusivamente em cookie HttpOnly, impedindo seu acesso por JavaScript. O token nunca é persistido em seu valor original no banco de dados, sendo armazenado como hash SHA-256. O mecanismo de rotação de sessão garante que a cada renovação os tokens anteriores sejam invalidados, limitando a janela de uso de um token comprometido.

A proteção contra SQL Injection é garantida pelo uso do Drizzle ORM, que abstrai a construção das consultas ao banco de dados e utiliza internamente prepared statements com parâmetros vinculados, eliminando a possibilidade de injeção direta de comandos SQL por meio das entradas da aplicação.

O risco de vazamento de tokens é mitigado pelo armazenamento do Access Token exclusivamente em memória no frontend, nunca em localStorage ou sessionStorage. O Refresh Token é transmitido apenas via cookie HttpOnly com as flags secure e sameSite configuradas para o ambiente de produção, reduzindo a exposição do token a scripts maliciosos e ataques de requisição forjada entre origens.

Contra-ataques de replay, o sistema aplica tempo de expiração curto ao Access Token, de 15 minutos, e invalida o código 2FA imediatamente após seu uso, removendo-o da tabela access_codes. O mecanismo de rotação de sessão também contribui para essa proteção, pois cada Refresh Token só pode ser utilizado uma única vez antes de ser revogado e substituído por um novo.

4. TESTES DE SEGURANÇA DOCUMENTADOS

Esta seção apresenta os testes de segurança realizados na aplicação, descrevendo a metodologia adotada, os cenários executados e os resultados obtidos. Os testes tiveram como objetivo validar o funcionamento correto dos mecanismos de segurança implementados, verificando se o sistema responde adequadamente a situações de uso indevido ou tentativas de acesso não autorizado.

4.1 Metodologia dos testes

Os testes de segurança foram realizados diretamente pela interface de documentação interativa da API, disponibilizada via Swagger no endpoint /docs da aplicação. Essa interface permite a execução de requisições HTTP aos endpoints da API de forma estruturada, possibilitando a simulação de diferentes cenários de uso, tanto válidos quanto inválidos.

A metodologia adotada consistiu na execução manual de casos de teste baseados nos fluxos documentados nas seções anteriores. Para cada cenário, foram definidos os dados de entrada, o comportamento esperado do sistema e o resultado obtido. Os testes foram organizados de forma a cobrir os principais pontos de controle de segurança da aplicação, incluindo autenticação, validação de tokens e controle de acesso.

4.2 Testes executados

Login inválido

O teste foi realizado submetendo credenciais incorretas na tela de login da aplicação. Ao informar e-mail ou senha inválidos, o sistema retornou o código HTTP 401 com a mensagem de credenciais inválidas, sem indicar qual dos campos estava incorreto, evitando assim a exposição de informações que pudessem auxiliar um possível atacante a identificar usuários cadastrados.



A imagem mostra a interface de login de uma aplicação web. No topo, o título "Login" é seguido pelo texto "Digite seu email e senha para continuar". Abaixo, há campos para "Email" e "Senha". O campo de email contém o texto "11231100756@alunos.umc.br" e o campo de senha contém pontos. Abaixo dos campos, uma mensagem de erro em vermelho diz "Credenciais inválidas". No centro, há um botão preto com o texto "Continuar". Na base da interface, há links para "Criar conta", "Esqueci a senha" e "Política de Privacidade".

Figura 13 – Credenciais inválidas

Tentativas de Brute force

Foram realizadas três tentativas consecutivas de login com senhas incorretas para um mesmo e-mail cadastrado. Após a terceira tentativa, o sistema bloqueou a conta temporariamente e retornou o código HTTP 423, confirmando o funcionamento do mecanismo de proteção. Novas tentativas durante o período de bloqueio foram igualmente rejeitadas.

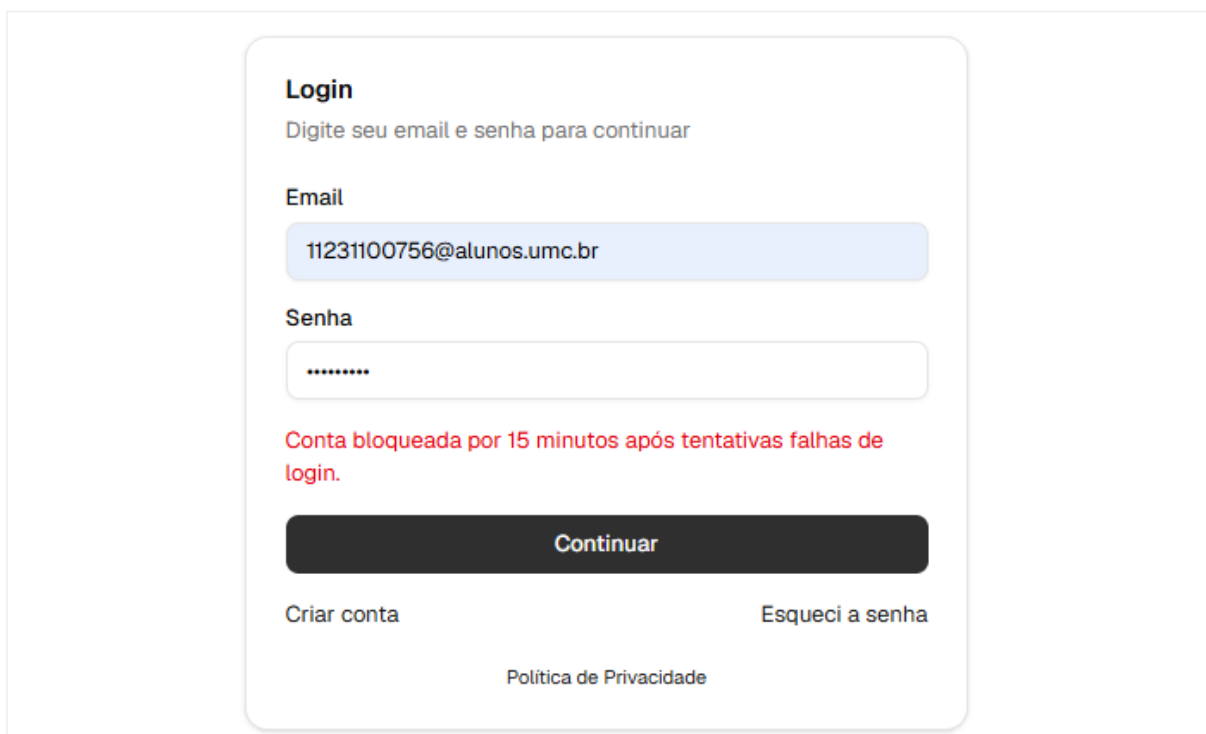


Figura 14 – Conta bloqueada após 3 credenciais inválidas

Validação de código de acesso – login

O teste foi realizado em duas etapas. Na primeira, após a validação bem-sucedida da senha informada pelo usuário, o sistema exibiu a tela de verificação de acesso, informando que um código temporário de 6 dígitos havia sido enviado ao e-mail cadastrado durante o processo de criação da conta. Essa etapa teve como objetivo validar o funcionamento do mecanismo de autenticação em dois fatores implementado pela aplicação.

Na segunda etapa, foi verificado o recebimento do e-mail enviado automaticamente pelo sistema com o remetente "UMC - Projeto de Segurança", contendo o código de acesso e a informação de que ele expiraria em 10 minutos por motivos de segurança. Após o recebimento, o código foi inserido corretamente na interface da aplicação e o acesso ao sistema foi concedido com sucesso. O teste confirmou o funcionamento adequado do fluxo de autenticação em duas etapas, bem como o envio correto do código temporário e a validação do token gerado pela aplicação.

Login

Digite o código de acesso enviado para seu email

Enviamos um código de 6 dígitos para o seu e-mail.

Código de Acesso

Digite o código

Entrar

Voltar

[Política de Privacidade](#)

Figura 15 – Solicitação de código de acesso

Código de verificação — login [Resumir este email](#)

U **UMC - Projeto de Segurança** <noreply@rotrenanoliveira.com>

Para: THAINA ESTHER SOARES DA SILVA

Olá, teste-umc.

Seu código de acesso para concluir o login é: SRNCO7

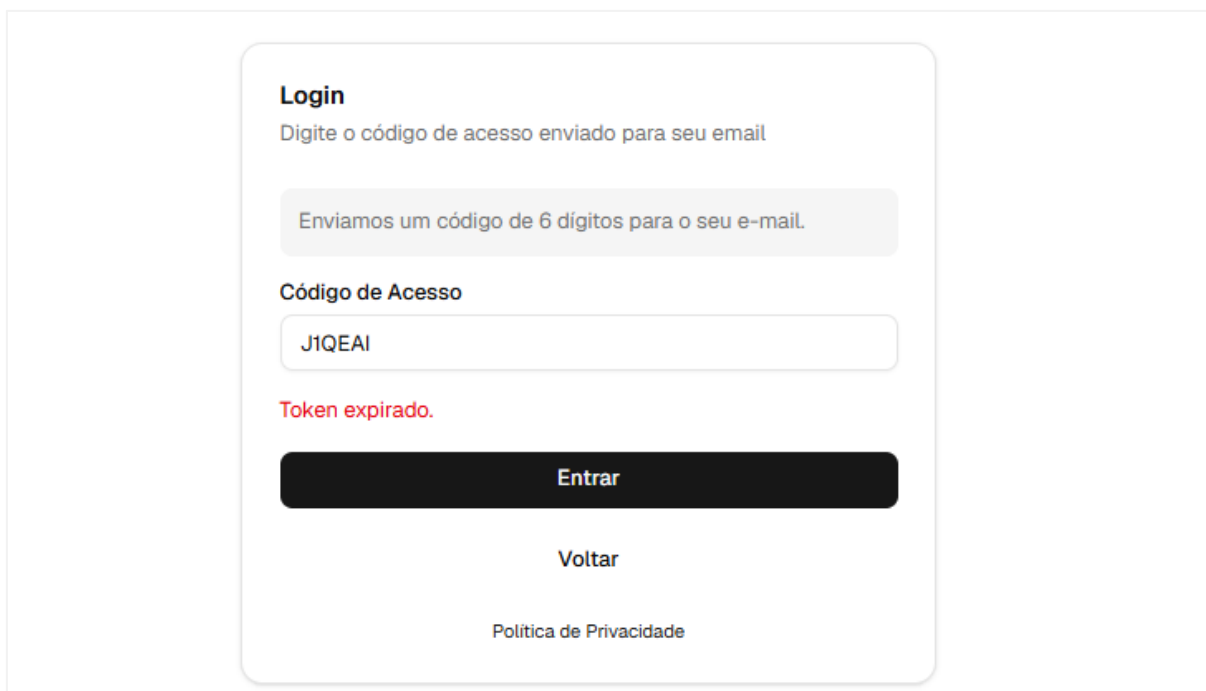
Ele expira em 10 minutos. Se você não tentou entrar, altere sua senha.

[← Responder](#) [→ Encaminhar](#)

Figura 16 – Envio de código de acesso ao e-mail

Expiração de token

O teste foi realizado aguardando o período de expiração do código de acesso 2FA, definido em 10 minutos. Ao tentar utilizar o código após esse prazo, o sistema retornou a mensagem de token expirado e manteve o usuário na mesma tela, impedindo o acesso sem redirecionar ou expor informações adicionais, confirmando o funcionamento correto do controle de expiração implementado.



Login
Digite o código de acesso enviado para seu email

Enviamos um código de 6 dígitos para o seu e-mail.

Código de Acesso

J1QEAI

Token expirado.

Entrar

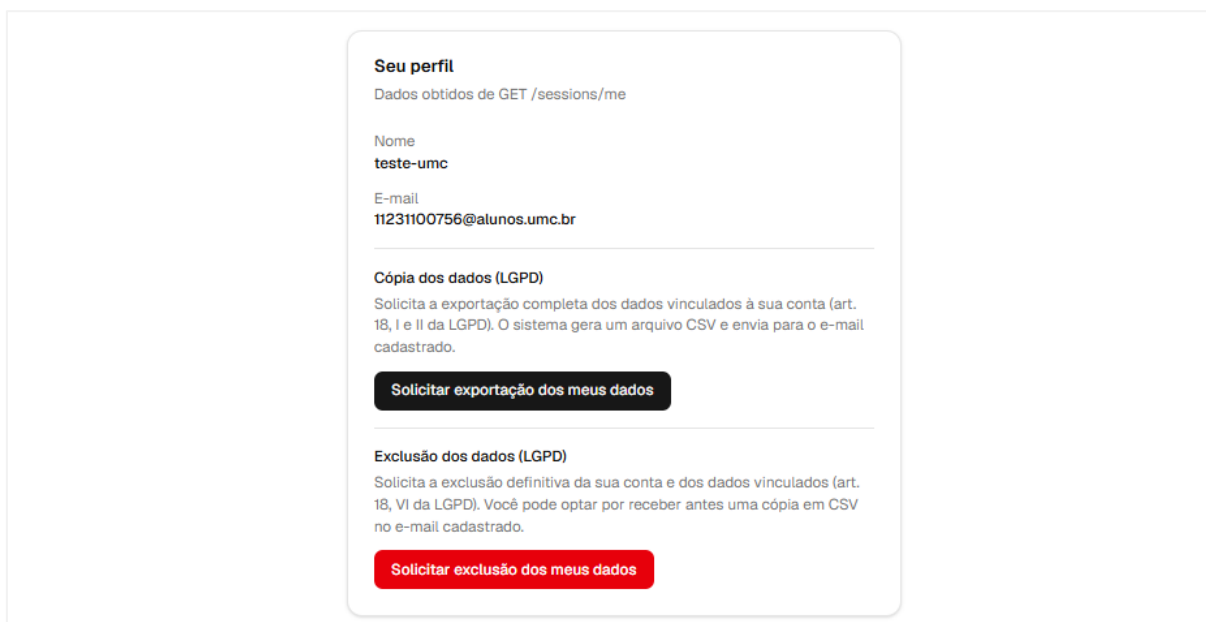
[Voltar](#)

[Política de Privacidade](#)

Figura 17 – Tentativa de login com token expirado

Verificação dos direitos do titular de dados (LGPD)

O teste foi realizado acessando a tela de perfil do usuário autenticado. O sistema exibiu corretamente os dados cadastrais do usuário, além das opções de exportação e exclusão dos dados pessoais previstas na LGPD.



Seu perfil
Dados obtidos de GET /sessions/me

Nome
teste-umc

E-mail
11231100756@alunos.umc.br

Cópia dos dados (LGPD)
Solicita a exportação completa dos dados vinculados à sua conta (art. 18, I e II da LGPD). O sistema gera um arquivo CSV e envia para o e-mail cadastrado.

Solicitar exportação dos meus dados

Exclusão dos dados (LGPD)
Solicita a exclusão definitiva da sua conta e dos dados vinculados (art. 18, VI da LGPD). Você pode optar por receber antes uma cópia em CSV no e-mail cadastrado.

Solicitar exclusão dos meus dados

Figura 18 - Tela de perfil do usuário com opções de direitos LGPD

Na solicitação de exportação, o sistema exibiu a mensagem confirmando o envio do arquivo CSV com os dados pessoais para o e-mail cadastrado. O e-mail foi recebido com o remetente "UMC - Projeto de Segurança", contendo o arquivo CSV

em anexo e a informação de que a exportação foi gerada conforme o artigo 18 da LGPD.

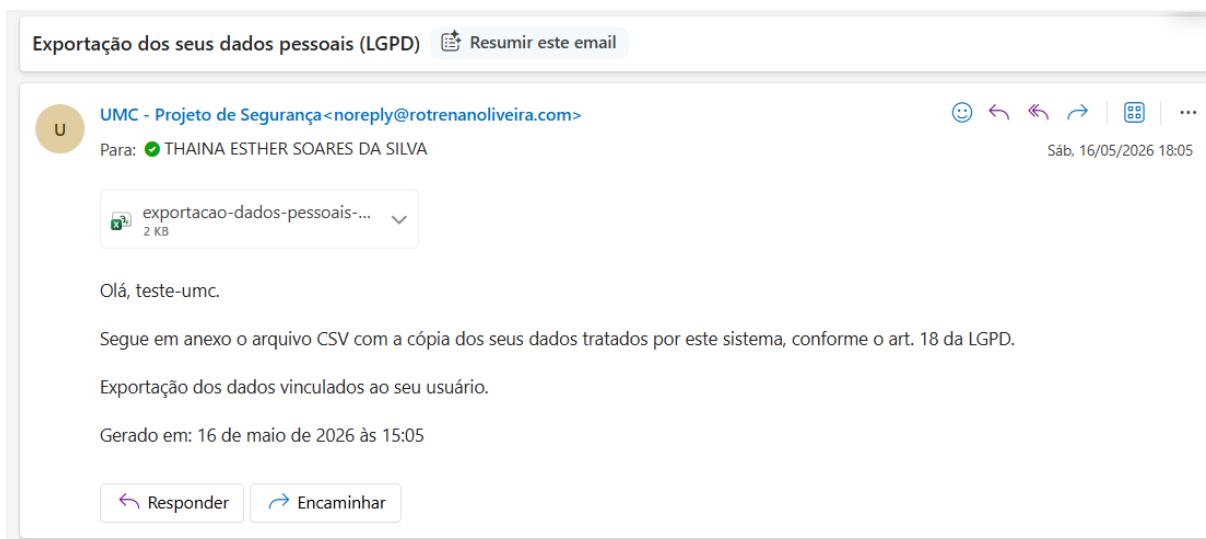


Figura 19 - E-mail de exportação dos dados pessoais conforme a LGPD

Na solicitação de exclusão, o sistema exibiu um modal de confirmação informando que a ação é irreversível e que a conta, códigos de acesso, sessões e registros associados seriam apagados. O modal também ofereceu a opção de receber uma cópia dos dados em CSV antes da exclusão, demonstrando o respeito ao direito do titular previsto no artigo 18, VI da LGPD.



Figura 20 - Modal de confirmação de exclusão de conta e dados

4.3 Resultados obtidos

Os testes realizados confirmaram o funcionamento correto dos mecanismos de segurança implementados na aplicação. Em todos os cenários de uso indevido simulados, o sistema respondeu conforme o esperado, negando o acesso e retornando os códigos de erro apropriados sem expor informações sensíveis nas mensagens de resposta da API. Nenhuma vulnerabilidade crítica foi identificada durante os testes, e os controles de bloqueio de conta, expiração de tokens e validação de credenciais operaram dentro dos parâmetros definidos no desenvolvimento.

Logs

Os logs gerados pelo servidor durante os testes evidenciam o comportamento esperado do sistema em cada cenário executado. Os registros mostram as requisições realizadas aos endpoints de autenticação, com os respectivos códigos de status retornados e os identificadores dos usuários envolvidos. É possível observar o fluxo completo de autenticação em duas etapas, com o POST /auth/authenticate-password retornando status 200 seguido do POST /auth/authenticate-access-code também com status 200, confirmando a conclusão bem-sucedida do login. As requisições ao endpoint GET /sessions/me com status 200 indicam sessões ativas e autenticadas. Também é registrado o acesso ao endpoint GET /sessions/me/personal-data com status 200, confirmando o funcionamento da funcionalidade de exportação dos dados pessoais prevista na LGPD. As entradas com status 204 referem-se a requisições do tipo OPTIONS, geradas automaticamente pelo navegador como parte do mecanismo de verificação de CORS antes das requisições principais.

logs	Plaintext
[2026-05-13T11:16:08.595Z]:POST:/auth/authenticate-password, status_code:200, made_by:JL79AHI4RSU0	
[2026-05-13T11:16:35.188Z]:POST:/auth/authenticate-access-code, status_code:200, made_by:JL79AHI4R	
[2026-05-13T11:18:56.139Z]:GET:/sessions/me, status_code:200, made_by:JL79AHI4RSU0 on IP:127.0.0.1	
[2026-05-13T11:21:53.965Z]:GET:/sessions/me, status_code:200, made_by:JL79AHI4RSU0 on IP:127.0.0.1	
[2026-05-13T11:22:00.559Z]:GET:/sessions/me/personal-data, status_code:200, made_by:JL79AHI4RSU0 o	
[2026-05-13T11:23:56.254Z]:OPTIONS:/auth/authenticate-password, status_code:204, made_by:null on I	
[2026-05-13T11:23:58.341Z]:POST:/auth/authenticate-password, status_code:200, made_by:null on IP:1	
[2026-05-13T11:24:10.870Z]:OPTIONS:/auth/authenticate-access-code, status_code:204, made_by:null o	
[2026-05-13T11:24:11.315Z]:POST:/auth/authenticate-access-code, status_code:200, made_by:null on I	
[2026-05-13T11:24:11.438Z]:OPTIONS:/sessions/me, status_code:204, made_by:null on IP:127.0.0.1	
[2026-05-13T11:24:11.443Z]:OPTIONS:/sessions/me, status_code:204, made_by:null on IP:127.0.0.1	
[2026-05-13T11:24:11.540Z]:GET:/sessions/me, status_code:200, made_by:JL79AHI4RSU0 on IP:127.0.0.1	
[2026-05-13T11:24:11.555Z]:GET:/sessions/me, status_code:200, made_by:JL79AHI4RSU0 on IP:127.0.0.1	

Figura 21 – Representação da saída do log

Respostas da API

A API do sistema é responsável por processar as requisições da aplicação e retornar respostas de acordo com as validações, regras de negócio e mecanismos de segurança implementados. Essas respostas seguem um padrão estruturado, contendo códigos de status HTTP, mensagens descritivas e informações relacionadas à operação realizada.

Os exemplos apresentados nesta seção demonstram diferentes cenários de autenticação e gerenciamento de usuários, incluindo cadastro de usuário, validação

de token JWT e bloqueio de conta após tentativas inválidas de acesso. Essas respostas permitem identificar o comportamento do sistema diante de situações de sucesso, validação e restrição de acesso, contribuindo para a segurança e o controle da aplicação.

As figuras a seguir apresentam exemplos visuais das respostas retornadas pela API durante a execução do sistema.

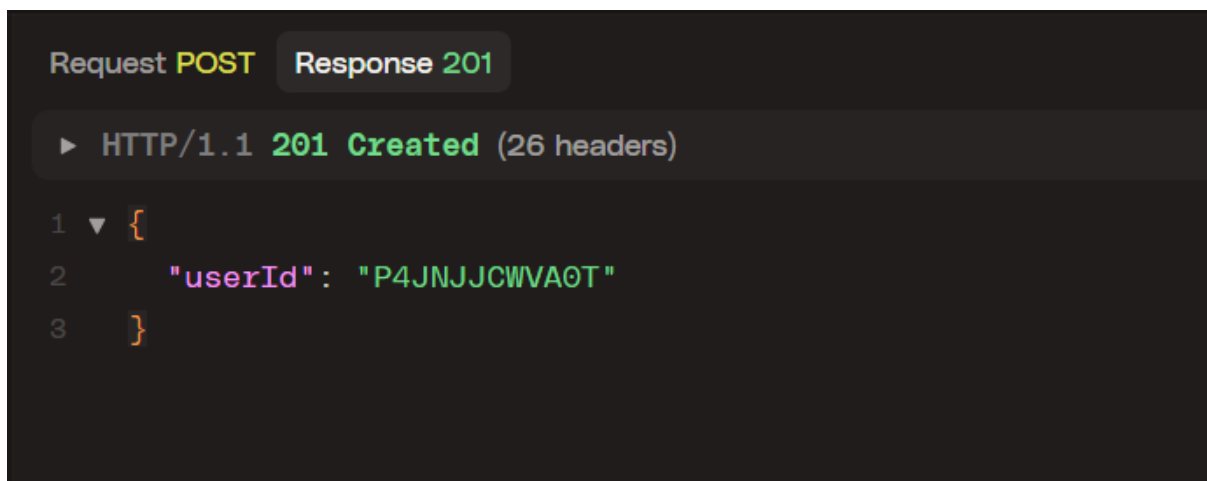


Figura 22 - Resposta da API no cadastro de usuário

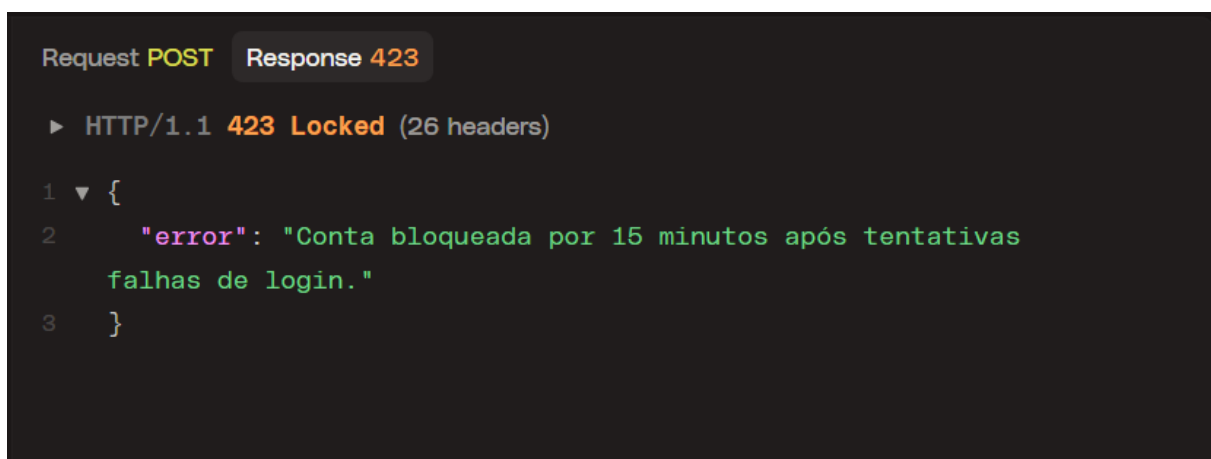


Figura 23 - Resposta de bloqueio de conta

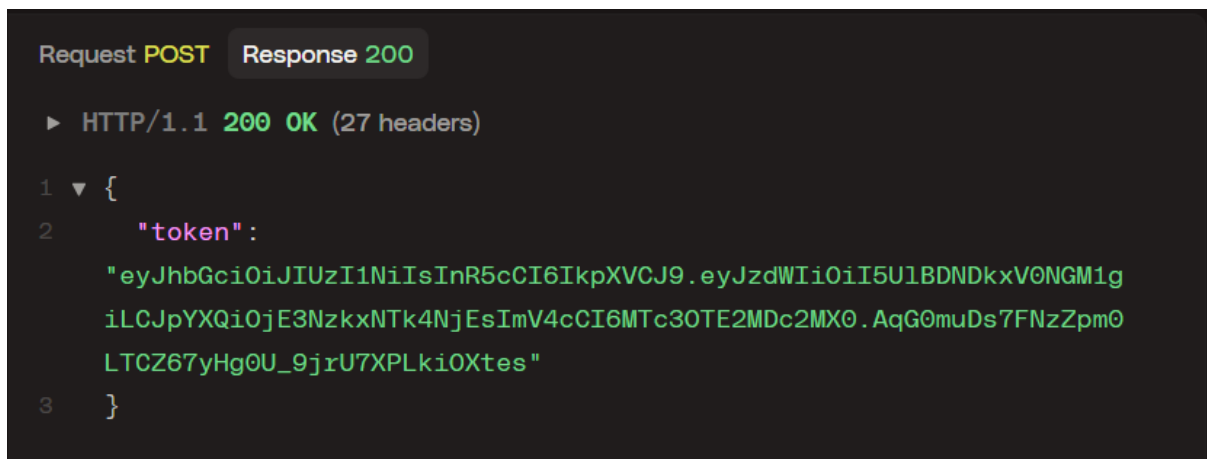


Figura 24 - Validação de token de autenticação

5. FUNDAMENTAÇÃO CIENTÍFICA

Esta seção apresenta os conceitos teóricos e normativos que fundamentaram as decisões de projeto e implementação da aplicação. Os temas abordados cobrem desde os princípios de autenticação e autorização até as legislações e normas técnicas aplicáveis ao contexto de segurança da informação.

5.1 Conceitos de autenticação e autorização

A autenticação é o processo pelo qual um sistema verifica a identidade de um usuário, confirmando que ele é quem afirma ser. Esse processo pode ser realizado por meio de diferentes fatores, como algo que o usuário sabe (senha), algo que ele possui (dispositivo ou token) ou algo que ele é (biometria). A autorização, por sua vez, é o processo que determina quais recursos e operações um usuário autenticado tem permissão de acessar. Enquanto a autenticação responde à pergunta "quem é você?", a autorização responde à pergunta "o que você pode fazer?".

No contexto da aplicação desenvolvida, a autenticação é realizada em duas etapas: a validação das credenciais do usuário por meio de e-mail e senha, seguida da verificação de um código temporário enviado ao e-mail cadastrado. A autorização é implementada por meio de tokens JWT, que carregam o identificador do usuário e são validados pelo servidor a cada requisição às rotas protegidas.

5.2 Funcionamento do JWT

O JSON Web Token (JWT) é um padrão aberto definido pela RFC 7519 que especifica um formato compacto e autocontido para transmissão segura de informações entre partes como um objeto JSON. Um token JWT é composto por três partes separadas por pontos: o cabeçalho (header), que especifica o algoritmo de assinatura utilizado; o payload, que contém as declarações (claims) sobre o usuário e metadados do token; e a assinatura, que garante a integridade do token e permite verificar que ele não foi alterado.

Na aplicação, o JWT é utilizado em dois contextos distintos. O Access Token é um JWT de curta duração, com expiração de 15 minutos, transmitido no corpo da resposta e utilizado pelo frontend no cabeçalho Authorization das requisições. O Refresh Token é um JWT de maior duração, com validade de 7 dias, transmitido exclusivamente via cookie HttpOnly e utilizado para renovar a sessão sem necessidade de novo login. Nenhum dos tokens carrega informações sensíveis no payload, sendo o identificador do usuário a única informação armazenada.

5.3 Criptografia de senhas com bcrypt

O bcrypt é uma função de hashing de senhas baseada no algoritmo de criptografia Blowfish, proposta por Niels Provos e David Mazières em 1999. Diferente de funções de hash genéricas como MD5 e SHA-1, o bcrypt foi projetado especificamente para o armazenamento seguro de senhas, incorporando dois mecanismos fundamentais de segurança: o fator de custo e o salt.

O fator de custo, configurável pela constante BCRYPT_ROUNDS na aplicação, determina o número de iterações do algoritmo, tornando o processo de hashing computacionalmente custoso. Isso dificulta ataques de força bruta e de dicionário, pois cada tentativa de adivinhar a senha exige um processamento significativo. O salt é um valor aleatório gerado automaticamente pelo bcrypt e incorporado ao hash resultante,

garantindo que duas senhas idênticas produzam hashes diferentes, o que impede o uso de tabelas pré-computadas conhecidas como rainbow tables. Na aplicação, o processo de salt é realizado internamente pelo bcrypt, não sendo necessário gerenciá-lo separadamente.

5.4 Conceitos de 2FA/MFA

A autenticação multifator (MFA) é uma técnica de segurança que exige que o usuário comprove sua identidade por meio de dois ou mais fatores independentes antes de obter acesso a um sistema. A autenticação em dois fatores (2FA) é o caso mais comum de MFA, combinando exatamente dois fatores distintos. Essa abordagem reduz significativamente o risco de acesso não autorizado, pois mesmo que um fator seja comprometido, o atacante ainda precisaria do segundo para concluir a autenticação.

A aplicação implementa o 2FA por e-mail, onde o segundo fator é um código numérico de seis dígitos enviado ao endereço de e-mail cadastrado do usuário após a validação bem-sucedida da senha. O código possui validade de 10 minutos e é removido do banco de dados imediatamente após seu uso, impedindo sua reutilização. Esse modelo é classificado como um fator de posse, pois pressupõe que somente o titular da conta tem acesso à caixa de entrada do e-mail cadastrado.

5.5 Segurança em APIs REST

As APIs REST (Representational State Transfer) são interfaces de comunicação amplamente utilizadas em aplicações web modernas para a troca de dados entre frontend e backend. Por serem expostas à internet, essas interfaces representam uma superfície de ataque relevante e devem ser protegidas por um conjunto de controles de segurança.

A aplicação adota diversas práticas recomendadas para a segurança de APIs REST. O plugin Helmet é utilizado para definir cabeçalhos HTTP de segurança, como o Content Security Policy e o X-Frame-Options, reduzindo a exposição a ataques comuns. O controle de CORS (Cross-Origin Resource Sharing) é configurado para aceitar requisições apenas de origens autorizadas, impedindo que aplicações externas não autorizadas consumam a API. O Rate Limit restringe o número de requisições por janela de tempo, protegendo os endpoints contra abusos e ataques automatizados. A validação de todas as entradas é realizada com a biblioteca Zod, garantindo que dados malformados ou inesperados sejam rejeitados antes de chegar à camada de negócio.

5.6 LGPD e proteção de dados

A Lei Geral de Proteção de Dados Pessoais (LGPD), instituída pela Lei nº 13.709 de 14 de agosto de 2018, estabelece regras sobre coleta, armazenamento, tratamento e compartilhamento de dados pessoais no Brasil. A lei define dados pessoais como qualquer informação relacionada a uma pessoa natural identificada ou identificável, e impõe às organizações a obrigação de tratar esses dados com base em princípios como finalidade, necessidade, transparência e segurança.

A aplicação adota medidas alinhadas aos princípios da LGPD. O consentimento do usuário é registrado no momento do cadastro por meio do campo consentAt, que armazena a data e hora do aceite dos termos de uso. O campo name do usuário é criptografado em nível de aplicação antes de ser persistido no banco de dados,

protegendo dados pessoais identificáveis. As senhas são armazenadas exclusivamente em formato de hash bcrypt, nunca em texto puro. O princípio da minimização de dados é seguido ao coletar apenas as informações estritamente necessárias para o funcionamento do sistema.

5.7 Referências acadêmicas, artigos e normas técnicas

OWASP

A Open Web Application Security Project (OWASP) é uma fundação sem fins lucrativos dedicada à melhoria da segurança de software. Seus guias e publicações são amplamente reconhecidos como referências técnicas na área de segurança de aplicações web. O OWASP Top 10 lista as vulnerabilidades mais críticas em aplicações web, e o Web Security Testing Guide (WSTG) fornece metodologias detalhadas para testes de segurança. As diretrizes da OWASP fundamentaram a identificação das ameaças descritas na Seção 3 e as contramedidas adotadas no sistema.

ISO 27001

A norma ISO/IEC 27001 é um padrão internacional que especifica os requisitos para estabelecer, implementar, manter e melhorar continuamente um Sistema de Gestão de Segurança da Informação (SGSI). Ela fornece um framework estruturado para a gestão de riscos de segurança da informação, cobrindo aspectos como controle de acesso, criptografia, segurança de operações e gestão de incidentes. Os princípios da norma orientaram a abordagem de segurança adotada no projeto, especialmente no que diz respeito ao controle de acesso e à proteção de dados sensíveis.

RFC JWT

A RFC 7519, publicada pelo Internet Engineering Task Force (IETF) em maio de 2015, define o padrão JSON Web Token (JWT). O documento especifica a estrutura do token, os algoritmos de assinatura suportados e as diretrizes para seu uso seguro em sistemas de autenticação e troca de informações. A RFC 7519 foi a referência técnica primária para a implementação do mecanismo de autenticação baseado em JWT na aplicação, incluindo a definição dos tempos de expiração e a estrutura dos claims utilizados.

Lei Geral de Proteção de Dados (LGPD)

A Lei nº 13.709, de 14 de agosto de 2018, conhecida como Lei Geral de Proteção de Dados Pessoais (LGPD), é a legislação brasileira que regula o tratamento de dados pessoais por pessoas físicas e jurídicas. Inspirada no Regulamento Geral de Proteção de Dados (GDPR) da União Europeia, a LGPD estabelece os direitos dos titulares de dados, as obrigações dos agentes de tratamento e as sanções aplicáveis em caso de descumprimento. A lei foi a referência normativa para as decisões de projeto relacionadas ao consentimento do usuário, à criptografia de dados pessoais e à minimização das informações coletadas pelo sistema.

6. REFERÊNCIAS

OWASP FOUNDATION. **Brute Force Attack**. Disponível em: https://owasp.org/www-community/attacks/Brute_force_attack. Acesso em: maio 2026.

OWASP FOUNDATION. **Blocking Brute Force Attacks**. Disponível em: https://owasp.org/www-community/controls/Blocking_Brute_Force_Attacks. Acesso em: maio 2026.

OWASP FOUNDATION. **Authentication Cheat Sheet**. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html. Acesso em: maio 2026.

OWASP FOUNDATION. **Session Hijacking Attack**. Disponível em: https://owasp.org/www-community/attacks/Session_hijacking_attack. Acesso em: maio 2026.

OWASP FOUNDATION. **SQL Injection**. Disponível em: https://owasp.org/www-community/attacks/SQL_Injection. Acesso em: maio 2026.

OWASP FOUNDATION. **Phishing**. Disponível em: <https://owasp.org/www-community/attacks/Phishing>. Acesso em: maio 2026.

OWASP FOUNDATION. **Testing for Session Hijacking**. Disponível em: https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/09-Testing_for_Session_Hijacking.html. Acesso em: maio 2026.

OWASP FOUNDATION. **Testing for Weak Lock Out Mechanism**. Disponível em: https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/04-Authentication_Testing/03-Testing_for_Weak_Lock_Out_Mechanism. Acesso em: maio 2026.

OWASP FOUNDATION. **Authorization Cheat Sheet**. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html. Acesso em: maio 2026.

OWASP FOUNDATION. **Password Storage Cheat Sheet**. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Acesso em: maio 2026.

OWASP FOUNDATION. **Multifactor Authentication Cheat Sheet**. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Multifactor_Authentication_Cheat_Sheet.html. Acesso em: maio 2026.

OWASP FOUNDATION. **REST Security Cheat Sheet**. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html. Acesso em: maio 2026.

OWASP FOUNDATION. **OWASP Top Ten**. Disponível em: <https://owasp.org/www-project-top-ten>. Acesso em: maio 2026.

OWASP FOUNDATION. **Web Security Testing Guide**. Disponível em: <https://owasp.org/www-project-web-security-testing-guide>. Acesso em: maio 2026.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: maio 2026.

JONES, M.; BRADLEY, J.; SAKIMURA, N. **RFC 7519: JSON Web Token (JWT)**. Internet Engineering Task Force (IETF), maio 2015. Disponível em: <https://www.rfc-editor.org/info/rfc7519>. Acesso em: maio 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR ISO/IEC 27001:2022**. Segurança da informação, segurança cibernética e proteção à privacidade: sistemas de gestão da segurança da informação: requisitos. Rio de Janeiro: ABNT, 2022.